Даня
@ka3ak_0ff

КОНСПЕКТ

Даня
@ka3ak_0ff

Правило суммы и правило произведения в комбинаторике

Комбинаторика — это раздел математики, изучающий способы подсчёта количества возможных вариантов выбора объектов при заданных условиях. Одними из ключевых принципов являются правило суммы и правило произведения.

Правило суммы

Если некоторый объект можно выбрать одним из n способов или одним из m способов, и при этом эти способы не пересекаются, то общее число способов выбрать объект равно n + m.

Пример: Представьте, что студенту нужно выбрать курс для обучения. Он может выбрать один из 5 курсов по математике или один из 3 курсов по информатике. Так как выбор делается либо в одной, либо в другой категории, общее количество вариантов равно 5 + 3 = 8.

Правило произведения

Если некоторый объект можно выбрать одним из n способов, а после этого другой объект можно выбрать одним из m способов, то общее число способов выбрать оба объекта равно n × m.

Пример: Допустим, в кафе на завтрак можно выбрать один из 3 видов напитков и одно из 4 блюд. Так как выбор напитка и выбор блюда независимы, общее количество возможных комбинаций завтрака равно 3 × 4 = 12.

Даня
@ka3ak_0ff

Перестановки

itertools.permutations — это функция из модуля itertools, которая генерирует все возможные перестановки элементов заданной последовательности.

Основные характеристики:

• Возвращает итератор с кортежами, содержащими перестановки элементов.

• Может принимать дополнительный аргумент r, задающий длину подмножеств перестановок.

• Если r не указан, то по умолчанию берутся перестановки всей последовательности.

```
import itertools
perms = itertools.permutations([1, 2, 3])
print(list(perms))

> [(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1,
2), (3, 2, 1)]
```

```
import itertools
perms = itertools.permutations("ABC", 2)
print(list(perms))

> [('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'C'),
('C', 'A'), ('C', 'B')]
```

Комбинаторное значение

Функция itertools.permutations вычисляет перестановки, что в комбинаторике соответствует формуле:

$$P(n, r) = \frac{n!}{(n-r)!}$$

Где:
- n — количество элементов в последовательности,
- r — длина перестановок.

Пример: Для n = 3 (элементы [1, 2, 3]) и r = 2:

$$P(3, 2) = \frac{3!}{(3-2)!} = \frac{3 \times 2 \times 1}{1} = 6$$

Мы получаем 6 возможных перестановок двух элементов.

Итог

- itertools.permutations позволяет эффективно находить все возможные упорядоченные подмножества элементов.

- Может применяться в задачах перебора, генерации паролей, анализа последовательностей и других алгоритмических задачах.

Даня
@ka3ak_0ff

Размещения

itertools.product — это функция из модуля itertools, которая генерирует декартово произведение входных последовательностей, эквивалентное вложенным циклам.

Основные характеристики:

•Возвращает итератор с кортежами, содержащими комбинации элементов из входных последовательностей.

•Позволяет задавать количество повторений элементов с помощью аргумента repeat.

•Используется для генерации всех возможных сочетаний с повторениями.

```
import itertools
prod = itertools.product([1, 2], ['A', 'B'])
print(list(prod))

> [(1, 'A'), (1, 'B'), (2, 'A'), (2, 'B')]
```

```
import itertools
prod = itertools.product("AB", repeat=2)
print(list(prod))

> [('A', 'A'), ('A', 'B'), ('B', 'A'), ('B', 'B')]
```

Комбинаторное значение

Функция itertools.product вычисляет размещения с повторениями, что в комбинаторике соответствует формуле:

$$A(n, r) = n^r$$

Где:
• n — количество элементов в последовательности,
• r — длина размещений.

Пример: Для n = 2 (элементы [A, B]) и r = 2:

$$A(2, 2) = 2^2 = 4$$

Мы получаем 4 возможных варианта пар из A и B.

Итог

•itertools.product позволяет находить все возможные комбинации элементов с повторением.

•Используется для генерации перестановок с повторениями, перебора параметров и моделирования всех возможных сочетаний в пространстве решений.

Даня
@ka3ak_0ff

Сочетания

itertools.combinations — это функция из модуля itertools, которая генерирует все возможные сочетания элементов заданной последовательности без повторений.

Основные характеристики:

• Возвращает итератор с кортежами, содержащими сочетания элементов.

• Не учитывает порядок элементов в комбинациях.

• Длина каждой комбинации задаётся аргументом r.

```
import itertools
comb = itertools.combinations([1, 2, 3], 2)
print(list(comb))

> [(1, 2), (1, 3), (2, 3)]
```

```
import itertools
comb = itertools.combinations("ABC", 2)
print(list(comb))

> [('A', 'B'), ('A', 'C'), ('B', 'C')]
```

Комбинаторное значение

Функция itertools.combinations вычисляет сочетания без повторений, что в комбинаторике соответствует формуле:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

Где:
• n — количество элементов в последовательности,
• r — длина сочетаний.

Пример: Для n = 3 (элементы [1, 2, 3]) и r = 2:

$$C(3, 2) = \frac{3!}{2!(3-2)!} = \frac{3\times2\times1}{2\times1\times1} = 3$$

Мы получаем 3 возможных сочетания: (1,2), (1,3), (2,3).

Итог

•itertools.combinations используется для генерации сочетаний без повторений.

•Позволяет перебирать подмножества заданной длины.

•Применяется в задачах выбора объектов, анализа данных и статистических вычислений.